五人小组分工与实施计划

《信号与系统（1）》课程项目 · 基于频域匹配滤波的乐音识别系统

版本 v1.0 丨 编制日期 2026-10-30 丨 适用对象：通信工程专业本科二年级 5 人小组 丨 交付周：第 15 周

目　录

0 一页速览：先看这 6 张表 2
1 论文技术底稿：我们到底要复现什么 3
2 系统总体设计：先定接口，再并行开工 5
3 五人分工表 8
4 各板块技术栈与学习路径 12
5 七周排期与阶段验收 21
6 五人高效合作建议 24
7 答辩准备 28
附录 A 论文关键数据（复现时直接对照） 29
附录 B 报告提交前格式检查清单（逐条打勾） 30
附录 C 每周周报模板（复制即用） 31

0 一页速览：先看这 6 张表

0.1 项目一句话定义

复现《基于频域匹配滤波的音频信号分析与识别》（赵凌览、李晓峰，《实验科学与技术》2018 年第 16 卷第 5 期，第 34–38 页）提出的乐音识别方法，最终交付一个**“对着麦克风弹几个音，屏幕立刻显示音名、时域分段和频谱”的桌面软件**，并用一组可复现的实验数据说明它在噪声环境下为什么还能认对。

一句话技术路线：**分帧 → 找音符起止 → Goertzel 取谐波谱线 → 能量归一化 → 26 路匹配滤波（相关） → 取最大值判决**。

0.2 为什么选这一篇（对应课程六个核心知识点，一个都不落）

课程核心知识	论文中的落点	我们的代码位置
卷积与零状态响应	匹配滤波器的输出 y[n]=s[n]∗h[n]，在 n=N−1 采样	`matcher.py` 第 3 步
系统函数 / 冲激响应	匹配滤波器 h_j[n]=S_j[N−1−n]，即 H(ω)=S*(ω)	`matcher.py` 模板翻转
频谱分析（傅里叶）	FFT 全谱对照、Goertzel 单谱线	`spectrum.py`
滤波与谐振器	Goertzel 是二阶 IIR 谐振器；短时能量是平方律检波+低通	`spectrum.py`、`vad.py`
零输入 / 零状态	递推滤波器初值为零即零状态响应；谐波衰减即系统自然响应	报告第 2 章推导
抽样定理	fs=16 kHz 的选取依据（Nyquist 8 kHz）	`audio_io.py`、报告 2.3

选它的额外好处：**算法链路短、每一步都能单独验收、不依赖 GPU、五个人的工作量天然可切分**，非常适合 7 周的大二小组。

0.3 必须记住的 8 个数字

数字	含义	出处
26	匹配滤波器路数 M，对应 B3–C6 共 26 个半音	论文表 1、3.2 节
2^(1/12) ≈ 1.05946	十二平均律相邻半音频率比	论文 1.1 节
246.94 / 1046.50 Hz	识别范围下界 B3 / 上界 C6	论文表 1
14.7 Hz	相邻半音最小频率间隔（B3–C4），决定分析窗长下限	由表 1 计算
16000 Hz	采样率，Nyquist=8 kHz	我们的参数约定
0.909–0.999	论文表 2 自相关对角线取值	论文表 2
4.5%	论文 SNR=0 dB 时错检率（200 次错 9 次）	论文表 4
90%	我们给自己定的乐句识别准确率下限	第 5 章验收表

0.4 五人角色速览

代号	角色	主责板块	一句话交付物	建议工作量
成员 1	组长 / 项目经理 / 集成负责人	板块 0 管理、板块 A 采集预处理、总集成、报告统稿	全组一条命令跑通全流程	21%
成员 2	时域算法 + 数据管理员	板块 B 端点检测与分割、测试集建设、板块 F 抗噪实验执行	一段录音被准确切成一个个单音	20%

代号	角色	主责板块	一句话交付物	建议工作量
成员 3	频域算法 + 理论担当	板块 C 频谱分析（Goertzel/FFT）、公式推导、报告第 2 章	每个单音变成 26 维特征向量	20%
成员 4	识别核心	板块 D 模板库与匹配滤波、板块 E 和弦分解	特征向量变成音名，且能给出相关度	21%
成员 5	系统与演示	板块 G 图形界面、板块 E 乐句与 MIDI 导出、板块 F 结果可视化、答辩 PPT	老师现场点两下就能看到识别结果	18%

> 工作量比例是**第 0 周的初值**，第 6 周版本冻结前按 Git 提交记录、任务卡完成情况复核调整一次，调整依据要留痕（见 6.8 节）。

0.5 七个关键日期与五道门

阶段	日期	门	必须达成的状态
第 1 周	10/31 – 11/06	**G0 立项门**	方案冻结、接口契约 v1.0、26 音模板数据就位
第 2 周	11/07 – 11/13	单点开发	三个人能各自画出自己的图
第 3 周	11/14 – 11/20	**G1 单音门**	单音端到端识别 ≥ 90%，复现论文表 2 相关矩阵
第 4 周	11/21 – 11/27	功能扩张	乐句识别跑通、GUI 骨架能录音
第 5 周	11/28 – 12/04	**G2 集成门**	20 音乐句 ≥ 85%，GUI 现场录音出结果，抗噪曲线出图
第 6 周	12/05 – 12/11	**G3 冻结门**	代码冻结、报告初稿完成、可执行文件打包成功
第 7 周	12/12 – 12/18	**G4 交付门**	彩排 2 次、材料提交、现场答辩

0.6 课程评分点与我们的对应交付物（照着打分表做项目）

评分维度（各 25%）	A 档要求	我们对应的交付物	责任人
功能实现	全部功能 + 令人满意的效果	26 音单音识别、三音和弦分解、20 音以上乐句识别、批量文件测试	成员 4
性能指标	达到指标 + **全面展示与分析**	错检率–SNR 曲线、混淆矩阵热图、消融对比（仅基频 vs 全模板）、单帧耗时基准	成员 2、成员 5
工程规范	**现场采集** + 运行通畅 + 界面方便	带麦克风实时录音的 GUI、一键启动脚本、打包 exe、README	成员 5
理论知识	清楚原理 + 正确回答提问	理论映射表（2.6 节）、公式推导、10 个答辩必答题预案（第 7 章）	成员 3

> 关键提醒：课程量表里“工程规范”的 A 档明确写了**“可现场采集信号”**。这一条只有 GUI + 麦克风能拿到，**必须做，不能只交离线脚本**。这是本计划里投入产出比最高的一个决定。

---

1 论文技术底稿：我们到底要复现什么

1.1 论文算法链路（六步，对应六个可独立验收的模块）

```
麦克风/文件 → ① 幅度归一化 → ② 分帧(25ms/10ms) → ③ 短时能量+过零率端点检测
                                                          ↓
```

```
                                  切割出单音片段
                                        ↓
④ Goertzel 在 f0, 2f0, …, 26f0 上取 26 根谱线 → 能量归一化 → 特征向量 S_i
                                        ↓
⑤ 26 路匹配滤波：h_j[n] = S_j[N-1-n]，在 n=N-1 采样 → r_ij = ⟨S_i, S_j⟩
                                        ↓
⑥ argmax 判决 → 音名；Top-3 → 和弦；逐段串接 → 乐句/MIDI
```

其中第 ⑤ 步是全文的理论核心：**匹配滤波在 n=N−1 时刻的采样输出，恰好等于两个频谱向量的内积，也就是相关系数**。这正是课程里“系统零状态响应 = 激励与冲激响应的卷积”的直接应用，必须在报告第 2 章把这条推导写透（推导模板见 2.6 节）。

1.2 论文给出的 4 个可复现实验

编号	论文位置	实验内容	论文结果	我们的复现目标
E1	图 2 / 图 4	C4# 的 FFT 连续谱 vs Goertzel 离散谱线	基频 277.18 Hz 有峰，二次谐波 554.36 Hz 峰值更大	图形形态一致，峰位误差 ≤ 1 个频率分辨单元
E2	表 2	C4–B4 七个白键的 7×7 相关矩阵	对角线 0.909–0.999，非对角 0.007–0.140	对角线 ≥ 0.85，非对角 ≤ 0.20
E3	表 3	三音和弦 C4–E4–G4 分解	E4 0.7062 / C4 0.5691 / G4 0.3763	Top-3 全部命中，相关度排序规律可解释
E4	表 4	乐句加高斯白噪声，SNR 20/15/10/5/0 dB	错检率 0 / 0 / 0 / 1.0% / 4.5%（各 200 次）	趋势一致，绝对数值允许偏差并分析原因

1.3 精读发现的 7 个待定点（这就是我们报告的“研究空间”）

把这 7 条写进报告 2.4 节“复现中的问题与处理”，是拿高分最省力的方式——它证明我们不是照抄，而是真的读懂了。

1. **序列长度 N 未明确定义。** 论文只说“长度为 N 的序列”，图 6 中采样点标为 n=26。若按 0 基索引则 N=27，若按 1 基索引则 N=26。我们的处理：统一取 26 根谐波（第 1–26 次），并在报告脚注说明这一约定，同时验证补零到 32 点不改变相关度（因为余弦相似度对补零不变）。
2. **“Goertzel 比 FFT 复杂度低”这一论断需要修正。** 若只需 K 根谱线、每根 O(N)，总代价 O(KN)；而 FFT 是 O(N $\log_2$N)，只有在 K < $\log_2$N 时 Goertzel 才更省。当 K=26、N=4096（$\log_2$N=12）时，Goertzel 的乘法次数反而更多。**Goertzel 真正的优势是**：(a) 可以精确落在任意频率 f0 上取样，不受 DFT 频点栅格限制，无需补零对齐；(b) 只需要 K 根谱线时不必计算和存储全谱，内存 O(1)；(c) 纯递推、无复数运算、易定点化。我们要把这三点讲清楚，而不是重复一句站不住的“更快”。**这一条如果答辩被问到，答对就是加分项，答错就是减分项。**
3. **标准音模板的来源未交代。** 是录音、软件合成还是标准音频文件？音色不同会不会失效？我们的处理：用 SoundFont 合成 + 真实钢琴采样 + 现场手机录音三套模板做音色鲁棒性对比，这是论文没做的增量实验。
4. **测试集偏小。** 表 2 只测了 7 个白键音符、每个 1 次。我们扩到 26 个半音 × 5 次录制 = 130 个样本，并给出混淆矩阵。
5. **和弦分解用 Top-K 排序存在原理缺陷。** 表 3 中 E4 的相关度（0.7062）反而高于 C4（0.5691），但这并不代表 E4 在和弦中能量最大——三音谐波列互相交叠，相关度不再等于贡献度。我们的改进：用**非负最小二乘（NNLS）**或**正交匹配追踪（OMP）**求解稀疏分解，对比 Top-K 的准确率。

6. **高音区谐波越过 Nyquist。** C6=1046.5 Hz 的第 8 次谐波已达 8.37 kHz > 8 kHz。若机械地取 26 根谐波，高音模板会有大量“空维度”，导致归一化后维度退化。我们的处理：有效谐波数取 min(26, ⌊8000/f0⌋)，超出部分补零，并在报告中讨论其对高音识别率的影响。
7. **端点检测的门限系数未给出。** 论文只说“通过短时能量和短时过零率判断”，具体门限、最小音长、最小间隔均未给出。这部分必须自己标定，也是我们工作量最大、最容易出问题的地方。

1.4 复现范围：明确做什么、不做什么

优先级	内容	说明
P0 必做	六步算法链路、26 音单音识别、乐句识别、论文表 4 抗噪曲线	不做完等于没复现，直接掉到 D 档
P1 应做	三音和弦分解、带麦克风的 GUI、26 音全量测试与混淆矩阵、与“仅取基频峰值”基线的消融对比	这些是把成绩从 B 拉到 A 的关键
P2 加分	NNLS 改进和弦分解、MIDI/五线谱导出、音色鲁棒性对比、Goertzel 与 FFT 的实测耗时对比、实时流式识别	选 1–2 项做透即可，不要贪多
不做	复音音乐自动转谱（多声部、踏板、颤音）、人声/多乐器识别、深度学习方案	明确写进报告“局限与展望”，反而显得清醒

1.5 与论文的差异点（报告的“创新性”就靠这一节）

1. **从离线文件到现场实时采集：**论文只处理已有音频文件，我们做成可现场录音的交互系统。
2. **从 7 个白键到 26 个半音全覆盖：**测试集扩大近 4 倍，并给出混淆矩阵。
3. **和弦分解算法改进：**NNLS 稀疏分解替代 Top-K 排序，用数据说明谁更准。
4. **复杂度论断的实证修正：**实测 Goertzel 与 FFT 在 K=26、N=4096 下的耗时，给出“什么时候该用 Goertzel”的判据。
5. **自动生成 MIDI：**识别结果直接落成可播放的文件，把“识别”闭环到“应用”。

---

2 系统总体设计：先定接口，再并行开工

2.1 模块划分（8 个板块）

板块	名称	负责人	核心产出文件	依赖
0	项目管理与集成	成员 1	`README.md`、看板、周报	无
A	音频采集与预处理	成员 1	`audio_io.py`、`preprocess.py`	无
B	端点检测与单音分割	成员 2	`vad.py`、`dataset/标注.csv`	A
C	频谱分析与特征提取	成员 3	`spectrum.py`、`frequency.py`	A
D	模板库与匹配滤波识别	成员 4	`template.py`、`matcher.py`	C
E	应用层：和弦 / 乐句 / MIDI	成员 4、成员 5	`chord.py`、`melody.py`、`export_midi.py`	B、D
F	性能评估与抗噪实验	成员 2、成员 5	`evaluate.py`、`figures/`	A–E 全部
G	图形界面与工程化	成员 5	`app.py`、`run.bat`、`dist/`	A–E 全部
H	报告与答辩	全员（成员 1 统稿）	`report.docx`、`答辩.pptx`	全部

依赖关系是**单向的**：A → B、A → C → D → E → F/G。这意味着第 1 周必须把 A 做出来，否则后面四个人全部堵住。这是全项目第一个、也是最容易被忽视的进度风险。

2.2 接口契约（第 1 周冻结，全组不得擅自修改）

并行开发能成立的前提，是每个人都在写同一份"数据结构约定"。 建议第 1 周由成员 1 写出 `types.py`，全组评审后打 tag `v0.1-contract`。之后任何人要改，必须提 Issue 并通知全部下游成员。

数据结构	字段	生产者 → 消费者	说明
`AudioClip`	`data: np.float32[L]`、`fs: int`、`source: str`	A → B, C, G	单声道、已归一化到 [−1,1]、16 kHz
`FrameMatrix`	`frames: np.float32[n_f, 400]`、`hop: int=160`、`fs: int`	A → B	分帧后的二维矩阵，n_f = ⌊(L−400)/160⌋+1
`NoteSegment`	`t_start: float`、`t_end: float`、`samples: np.float32[]`、`index: int`	B → C, E	时间单位**秒**，样本为原始归一化波形
`SpectrumVector`	`note: str`、`midi: int`、`bins: np.float32[26]`、`freqs: np.float32[26]`	C → D	bins 已做 L2 能量归一化，$\|bins\|_2=1$
`MatchResult`	`top1: str`、`scores: np.float32[26]`、`margin: float`、`accepted: bool`	D → E, G, F	margin = 第一名减第二名，用于拒识

三条硬约束（写进 README，违反者代码不予合入）：

1. 所有函数**只接受上面 5 个结构体或基础类型**，不允许在模块之间传递裸 `dict`。
2. 时间戳一律用**秒（float）**，样本索引一律用**整数**，禁止混用。
3. 每个模块必须提供一个 `demo_xxx.py`，能独立运行并画出一张图。**没有 demo 的模块视为未完成。**

2.3 关键技术参数表（全组统一的"参数宪法"）

这张表每改一次都要全组通知，因为它会同时影响四个人的输出结果。

参数	取值	依据 / 说明
采样率 fs	16000 Hz	钢琴最高基频 C6=1046.5 Hz，识别所需上限约 8 kHz；16 kHz 恰好覆盖且算力只有 44.1 kHz 的 36%
位深 / 声道	16 bit / 单声道	乐音识别不依赖立体声
端点检测帧长	25 ms（400 点）	语音/乐音短时平稳假设为 10–30 ms
端点检测帧移	10 ms（160 点）	重叠 62.5%，时间分辨率 10 ms
端点检测窗	Hamming	主瓣较窄、旁瓣 −43 dB，比矩形窗泄漏小
能量高门限	T_high = 5 × E_noise	需用实测数据标定，此处为起点
能量低门限	T_low = 2 × E_noise	双门限防止音符被切碎
最小音符时长	80 ms	钢琴快速连奏最短约 60–80 ms
最小静音间隔	50 ms	小于此值视为同一音符内部起伏
频谱分析窗长	整个音符段，且 ≥ 150 ms	相邻半音最小间隔 14.7 Hz（B3–C4）→ 理论下限 1/14.7≈68 ms，留 2 倍余量
FFT 点数	$N_{fft} = 2^{\lceil \log_2 L \rceil}$	补零到 2 的幂，便于 FFT；仅用于对照绘图
谐波根数 K	26（有效数 = min(26, ⌊8000/f0⌋)）	对应论文 N=26/27 的约定
识别音域	MIDI 59–84（B3–C6），共 26 个半音	论文表 1
判决规则	argmax + 阈值 τ=0.5 + margin 拒识	论文式 (4)，阈值需标定
和弦输出	Top-3 + 阈值 0.25	论文 4.1 节
抗噪 SNR 档位	20 / 15 / 10 / 5 / 0 dB	论文表 4
每档检测次数	10 轮 × 20 音符 = 200 次	与论文一致，保证可比
随机种子	2026（固定）	保证抗噪实验可复现

2.4 建议的代码目录结构

```
project/
├─ README.md                # 环境安装、运行方法、模块说明（成员 1）
├─ requirements.txt         # 依赖清单（成员 1）
├─ types.py                 # 5 个数据契约（成员 1，冻结）
├─ config.py                # 2.3 节全部参数常量（成员 1）
├─ audio_io.py              # 板块 A：录音/读文件/重采样/归一化
├─ preprocess.py            # 板块 A：分帧/加窗
├─ vad.py                   # 板块 B：短时能量、过零率、双门限状态机
├─ frequency.py             # 板块 C：十二平均律频率表、MIDI 换算
├─ spectrum.py              # 板块 C：FFT、Goertzel、谐波拾取
├─ template.py              # 板块 D：模板生成/加载/能量归一化
├─ matcher.py               # 板块 D：26 路匹配滤波、判决、拒识
├─ chord.py                 # 板块 E：Top-K 与 NNLS 和弦分解
├─ melody.py                # 板块 E：乐句串接、编辑距离比对
├─ export_midi.py           # 板块 E：MIDI 导出
├─ evaluate.py              # 板块 F：awgn、SNR 扫描、错检率、混淆矩阵
├─ app.py                   # 板块 G：GUI 主程序
├─ demo_audio.py 等          # 每个模块一个可独立运行的 demo
├─ tests/                   # pytest 单元测试（每模块 ≥ 3 个）
├─ data/                    # 音频与模板（不入 Git，用脚本重建）
├─ figures/                 # 报告用图（300 dpi PNG 或 PDF）
└─ report/                  # 报告与答辩材料
```

2.5 课程理论与项目的映射表（答辩时直接照着讲）

课程概念	在项目中的具体形态	关键公式	报告位置
抽样定理	fs=16 kHz 的选择	fs > 2f_max	2.1
离散卷积	匹配滤波器输出	y[n]=Σ_k S_i[k]·h_j[n−k]	2.4
零状态响应	滤波器初值为 0 时的输出	y_zs[n] = x[n] ∗ h[n]	2.4
系统函数	匹配滤波器 H(ω)=S*(ω)	h[n]=S_j[N−1−n] ⇔ H(ω)=S_j*(ω)e^{−jω(N−1)}	2.4
相关与匹配	n=N−1 处采样 = 内积	y[N−1]=Σ_k S_i[k]S_j[k]=r_ij[0]	2.4
Cauchy–Schwarz	自相关最大性的证明	ΣS_i² − ΣS_iS_j = ½Σ(S_i−S_j)² ≥ 0（在 ‖S‖=1 下）	2.4
二阶 IIR 谐振器	Goertzel 递推	H(z)=(1−e^{−jω₀}z^{−1})/(1−2cosω₀·z^{−1}+z^{−2})	2.5
DFT / 频谱泄漏	有限长截断导致能量外泄	窗函数主瓣宽度 ∝ 1/N	2.5
滤波（平方律检波+低通）	短时能量	E_n=Σ_m[x(m)w(n−m)]²	2.3
滤波（差分+取绝对值+低通）	短时过零率	Z_n=½Σ_m\	sgn x(m)−sgn x(m−1)\
信噪比定义	awgn 实验	SNR_dB=10log₁₀(Σx²/Σn²)	3.4

> **这张表是本项目最值钱的一页纸。** 课程“理论知识”占验收 25%，老师问的每一个问题几乎都能在这张表里找到出处。建议打印出来贴墙上，每人背熟自己那三行。

2.6 关键推导模板（报告 2.4 节直接可用）

设待识音频的归一化频谱序列为 S_i[n]，第 j 路匹配滤波器的冲激响应为 h_j[n]=S_j[N−1−n]，则零状态响应在 n=N−1 处的采样值为

```
y_j[N-1] = Σ_{k=0}^{N-1} S_i[k] · h_j[N-1-k]
         = Σ_{k=0}^{N-1} S_i[k] · S_j[k]                    … (1)
         = ⟨S_i, S_j⟩ = r_ij[0]
```

即匹配滤波在终端时刻的采样输出**等价于两频谱序列的内积**。当 ‖S_i‖₂=‖S_j‖₂=1 时，

```
r_ii − r_ij = Σ S_i²[n] − Σ S_i[n]S_j[n]
            = ½ Σ (S_i[n] − S_j[n])² ≥ 0                    … (2)
```

故输入信号与自身模板的相关度恒不小于与其他模板的相关度，判决规则 argmax_j r_ij 是**最优**的（在等能量、高斯白噪声假设下等价于最大似然判决）。式 (2) 的验证同时也是我们单元测试的一项内容。

---

3 五人分工表

3.1 总分工表（主责唯一、兼责互补，每人都有一条完整的“从输入到输出”的链路）

代号	姓名	角色	主责板块	兼责板块	核心交付物	工作量
1	_____	组长 / 项目经理 / 集成负责人	板块 0 项目管理、板块 A 采集与预处理	报告统稿、Git 仓库、README	`types.py` `config.py` `audio_io.py` `preprocess.py` `README.md`	21%
2	_____	时域算法 + 数据管理员	板块 B 端点检测与单音分割	测试集建设、抗噪实验执行	`vad.py`、标注文件、测试集、`SNR` 实验数据	20%
3	_____	频域算法 + 理论担当	板块 C 频谱分析与特征提取	报告第 2 章公式、答辩理论题	`spectrum.py` `frequency.py`、公式推导、理论映射表	20%
4	_____	识别核心	板块 D 模板库与匹配滤波	和弦分解（与成员 5 分工）	`template.py` `matcher.py` `chord.py`、相关矩阵	21%
5	_____	系统与演示	板块 G 图形界面	乐句/MIDI 导出、评估可视化、答辩 PPT	`app.py` `melody.py` `export_midi.py`、GUI、PPT	18%

为什么这样分？ 三条理由：

1. **沿数据流切分，不沿“技术名词”切分。** 每个人负责一段“输入进去、输出出来”的完整环节（波形→帧→音符段→频谱向量→音名→界面），而不是“你学傅里叶、我学卷积”这种无法验收的分法。
2. **每人都有一个“难”的核心和一块“容易”的配套。** 成员 3、成员 4 承担算法难度，成员 2、成员 5 承担工程与数据量，成员 1 承担协调与集成，难度大致拉平。
3. **每人都能被别人替代。** 见 6.5 节“结对备份”：成员 1 与 5 互为备份（都在系统层），成员 2 与 3 互为备份（都在信号处理层），成员 4 由 1 兜底。

3.2 成员 1：组长 / 项目经理 / 集成负责人

职责定位：让五个人同时有效工作，并保证最后能合成一个能跑的东西。你的产出不是“最多的代码”，而是“清晰的约定和持续的推进”。

主责内容

1. **第 1 周必须完成的四件事**（这四件事没做完，全组都无法开工）：

- 写 `types.py`（5 个数据契约）和 `config.py`（2.3 节全部参数常量）；

- 写 `audio_io.py`：16 kHz 单声道录音、读 WAV、重采样、幅度归一化；
- 写 `preprocess.py`：分帧、加窗，输出 `FrameMatrix`；
- 写 `README.md`：环境安装步骤（`pip install -r requirements.txt`）、目录说明、如何跑 demo。

2. **集成与联调**：维护 `main_demo.py` 主流程（录音 → VAD → 频谱 → 匹配 → 输出），在 G1、G2、G3 三道门上做集成测试。
3. **项目管理**：看板维护、每周站会、每周日周报（发给组员，第 4 周起抄送助教）、里程碑提醒、风险登记册。
4. **对外沟通**：第 1 周内找任课教师/助教确认三件事——具体验收日期、是否允许 Python、报告是否有额外格式要求。

兼责内容：报告统稿（把 4 个人写的章节统一术语、编号、图表格式，这一步至少留 8 小时）；答辩开场与系统演示讲解。

交付清单：`types.py`、`config.py`、`audio_io.py`、`preprocess.py`、`main_demo.py`、`README.md`、`requirements.txt`、周报 7 份、会议记录 7 份。

能力要求：能读懂别人的代码；会用 Git 的 merge 和冲突解决；能画出清晰的框图；有耐心做琐事。**技术最强不是必要条件，靠谱才是。**

最大风险：把自己变成“包揽一切的组长”。见 6.4 节的三条边界。

3.3 成员 2：时域算法 + 数据管理员

职责定位：把一段连续录音，准确地切成一个个单音。**这是全项目对最终准确率影响最大的一环**——切错一个音，后面再准也没用。

主责内容

1. **板块 B：端点检测与单音分割**

- 计算短时能量 E_n 与短时过零率 Z_n（公式见 2.5 节）；
- 噪声底估计（取录音前 200 ms 的 20% 分位数）、双门限判决；
- 四状态机（静音 → 疑似起点 → 音符内 → 疑似终点 → 静音），带最小音长与最小间隔约束；
- 后处理：能量曲线中值滤波（5 帧）、起点回退 10 ms（保证吃进钢琴的击键瞬态）。

2. **测试集建设**（第 2–3 周，与算法开发并行）

- 26 个半音 × 5 次 = 130 个干净单音样本，统一命名 `note_C4_01.wav`；
- 20 音《玛丽有只小羊羔》乐句 3 条（SoundFont 合成 1 条、手机钢琴 App 1 条、真实钢琴/电子琴录音 1 条）；
- 三音和弦 C4-E4-G4 各 5 次；
- **人工标注每个音符的起止时间**，存成 `dataset/labels.csv`（这是评估端点检测的唯一依据，必须严谨）。

3. **板块 F 的抗噪实验执行**：调用 `evaluate.py` 的 SNR 扫描，跑完 5 档 × 200 次，填表并画曲线。

兼责内容：数据版本管理（`make_dataset.py` 一键重建数据集）；报告第 3.2 节“端点检测结果”。

交付清单：`vad.py`、`demo_vad.py`、`dataset/labels.csv`、`make_dataset.py`、抗噪实验原始数据 CSV。

能力要求：理解“短时分析”和“窗函数”；会用 `scipy.ndimage.median_filter`。**这份工作一半是算法、一半是耐心和细心**，适合做事踏实的同学。

最大风险：端点检测调参陷入无限循环。**对策：第 3 周 G1 门之后冻结参数，之后只允许改后处理逻辑，不允许再动门限系数**，否则抗噪实验的数据全部作废。

3.4 成员 3：频域算法 + 理论担当

职责定位：把“一个音符的一段波形”变成“一个 26 维的频谱特征向量”，并把背后的数学讲清楚。

主责内容

1. **板块 C：频谱分析**

- `frequency.py`：十二平均律频率表（`f = 440 × 2^((midi-69)/12)`）、音名与 MIDI 号互转、26 个半音的频率数组；
- `spectrum.py` 的 FFT 分支：加窗、`np.fft.rfft`、幅度谱、峰值检测（`scipy.signal.find_peaks` 的 `height` / `prominence` / `distance` 参数）；
- `spectrum.py` 的 Goertzel 分支：二阶 IIR 递推实现，在 f0, 2f0, …, 26f0 上精确取 26 根谱线；
- **交叉验证**：同一频点上，Goertzel 结果与 FFT 结果相对误差 < 1e-9；
- 高音区越界处理：有效谐波数 min(26, ⌊8000/f0⌋)，越界补零。

2. **理论推导与撰写**

- 报告 2.4 节：匹配滤波与相关的等价性推导（模板见 2.6 节）；
- 报告 2.5 节：Goertzel 的谐振器解释、DFT 栅栏效应与频谱泄漏；
- 报告 2.1 节：抽样率选取依据；
- 制作 2.5 节“理论映射表”，并在第 6 周给全组讲一次（40 分钟），确保每个人都能回答自己模块的问题。

兼责内容：第 7 章 10 个答辩必答题的理论部分；报告公式排版（统一用公式编辑器，连续编号）。

交付清单：`frequency.py`、`spectrum.py`、`demo_spectrum.py`、C4# 的 FFT 与 Goertzel 对照图（复现论文图 2/图 4）、报告 2.1/2.4/2.5 节初稿、理论串讲 1 次。

能力要求：能读懂 DFT 定义与卷积定理；会推导匹配滤波器的冲激响应；Python 的复数与数组运算熟练。**这是全组数学要求最高的岗位**，建议由信号与系统课程成绩较好的同学担任。

最大风险：只调库不推导，答辩被问“Goertzel 的系统函数是什么”时答不上来。**对策：成员 3 必须在第 5 周前手推一遍 H(z) 并在群里发照片留痕。**

3.5 成员 4：识别核心

职责定位：把 26 维特征向量变成音名。这是论文的灵魂，也是最容易被追问的地方，**必须理解到能徒手画出示意图的程度**。

主责内容

1. **板块 D：模板库与匹配滤波**

- 26 个标准音模板的获取与清洗：SoundFont 合成（`fluidsynth` + `FluidR3_GM.sf2`）为主，真实采样为辅；

- 每个模板走完整链路得到 S_j[26]，做 L2 能量归一化（$\|S_j\|_2=1$）；
- **实现 26 路匹配滤波**：h_j[n]=S_j[N−1−n]，用矩阵乘法一次算完 26 路（`R = S_unit @ T_unit.T`），禁止写 26 次 for 循环；
- 判决：argmax + 阈值 τ + margin 拒识（第一名与第二名差值过小则标“不确定”）；
- **复现论文表 2** 的 7×7 相关矩阵，对角线 ≥ 0.85、非对角 ≤ 0.20，并对 C4–G4 之间约 0.1 的相关度给出解释（纯五度 3:2，G4 恰好是 C4 的第 3 次谐波）。

2. **板块 E 之一：和弦分解**

- Top-K 实现（论文方法）；
- **改进版**：非负最小二乘 `scipy.optimize.nnls` 或 OMP 稀疏分解，与 Top-K 做准确率对比；
- 复现论文表 3，并解释为什么 E4 的相关度高于 C4。

兼责内容：报告 3.3 节“单音识别与和弦分解结果”；答辩时主讲算法原理。

交付清单：`template.py`、`matcher.py`、`chord.py`、`demo_match.py`、7×7 相关矩阵热图、和弦分解对比表、报告 3.3 节初稿。

能力要求：理解“卷积 = 相关（当滤波器翻转时）”；理解 Cauchy–Schwarz 不等式为什么保证自相关最大；矩阵运算熟练（避免 26 次循环）。

最大风险：模板音色与测试音色差异太大导致准确率上不去。**对策：第 3 周就做“模板-测试”交叉实验（合成模板测真实录音），把音色鲁棒性作为独立问题提前暴露，而不是等到第 5 周。**

3.6 成员 5：系统与演示

职责定位：让老师在三分钟内看懂我们做了什么。**课程量表里“工程规范”的 A 档要求“可现场采集信号 + 界面操作方便”，这一条完全由你负责。**

主责内容

1. **板块 G：图形界面**（第 4 周开始，不要等到最后）

- 主界面四区：左上波形+分段标注、右上频谱、左下识别结果（大字音名+钢琴卷帘）、右下参数面板；
- 功能按钮：开始录音 / 停止 / 打开文件 / 识别 / 导出 MIDI / 批量测试；
- 录音放在 `QThread` 里，避免界面卡死；
- 参数面板可调：帧长、门限系数、SNR（用于现场演示抗噪效果）；
- 一键启动：`run.bat` 或打包 `dist/app.exe`。

2. **板块 E 之二：乐句与导出**

- `melody.py`：逐段识别串接成序列，用编辑距离（`difflib.SequenceMatcher`）与标准乐谱比对，算准确率；
- `export_midi.py`：用 `mido` 或 `pretty_midi` 导出 MIDI，能被 Windows 播放器/MuseScore 正确打开。

3. **板块 F 的可视化**：错检率–SNR 曲线、混淆矩阵热图、耗时对比柱状图（成员 2 出数据，你出图）。

兼责内容：答辩 PPT（12–15 页）、90 秒演示脚本、现场演示应急预案。

交付清单：`app.py`、`melody.py`、`export_midi.py`、`run.bat`、`dist/`、报告全部图表（300 dpi）、PPT、演示视频 1 段（备份用）。

能力要求：会用 PySide6 或 Streamlit 做界面；会用 matplotlib 嵌入 Qt（`FigureCanvasQTAgg`）；有审美，知道什么样的图能让老师一眼看懂。

最大风险：GUI 打包失败（`sounddevice` 和 `soundfile` 的二进制依赖是经典坑）。**对策：第 5 周就做一次打包实验，失败就立刻退回"源码 + `run.bat` + 环境说明"方案，不要拖到第 7 周。**

3.7 分工会不会不均？——工作量自检

成员	算法难度	代码量（估）	文档量	协调成本	综合
1	中	500 行	高（统稿）	**很高**	21%
2	中	450 行	中	中（数据协调）	20%
3	**高**	400 行	**高**（公式）	低	20%
4	**高**	450 行	中	中	21%
5	中	600 行	中（PPT）	中	18%

全项目代码量约 2400 行（含 demo 与测试），文档约 1.2 万字，7 周 × 5 人 × 每周 8–10 小时 ≈ 300–350 人时。**这个量是合适的：有余量做改进点，但不多到做不完。**

> 如果第 2 周发现有人明显吃力（比如完全跟不上 `numpy` 向量化），**立刻启动 6.5 节的结对机制，而不是拖到第 5 周换人**。换人在这个阶段成本极高。

---

4 各板块技术栈与学习路径

> **使用说明**：每位成员只需精读自己的那一节 + 4.0 节。4.0 节是全员第 1 周的共同作业，**不完成不许开自己的模块**。每节的"最小验证实验"完成后截图发群，视同该阶段任务完成。

4.0 全员公共技术栈（第 1 周，5 人同时完成，约 6 小时）

要学的东西

知识点	具体到能做什么	建议用时
Python 基础语法	会写函数、类的 `__init__`、`if __name__`、列表推导、异常处理	已有基础可跳过
numpy 数组与向量化	会切片、广播、布尔索引、`np.concatenate`、`np.dot` / `@`；理解"避免 for 循环遍历数组"	3 小时
matplotlib 画图	会画 `plot` / `bar` / `imshow`，会设标题、坐标轴、`grid`、`legend`、保存 `savefig(dpi=300)`；**会显示中文**	1 小时
Git 基础	`clone` / `add` / `commit` / `push` / `pull` / `branch` / `checkout` / 解决一次冲突	1 小时
信号与系统复习	DFT 定义式、采样定理、卷积定义、系统函数 H(jω) 的含义	1 小时

必备库清单（写进 `requirements.txt`，版本锁定）

```
numpy>=1.26
scipy>=1.11
matplotlib>=3.8
soundfile>=0.12
sounddevice>=0.4.6
pandas>=2.0
PySide6>=6.6          # 成员 5 必装，其他人可不装
mido>=1.3             # 成员 5
pytest>=7.4           # 全员
```

matplotlib 显示中文的三行代码（这是新手第一大坑，直接抄）

```
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False   # 负号显示
```

最小验证实验：画一个 440 Hz、1 秒的正弦波，横轴带“时间/s”中文标签，保存为 `test_440hz.png`。5 个人都要能独立跑通——这一步不通，后面所有图都会是乱码。

4.1 板块 A：音频采集与预处理（成员 1）

要学的理论

- 抽样定理：fs > 2f_max。我们的识别上限约 8 kHz，故 fs=16 kHz。
- 幅度归一化为什么不影响识别：频谱是线性变换，`S(ax) = a·S(x)`，而后续做的是 **L2 归一化后的余弦相似度**，标量因子在分子分母中抵消。
- 分帧与短时平稳性：乐音在 10–30 ms 内可近似为平稳信号。
- 窗函数：矩形窗旁瓣高（−13 dB）导致泄漏，Hamming 窗旁瓣 −43 dB，主瓣稍宽。**权衡：主瓣宽→频率分辨率下降，旁瓣低→泄漏小。**

要用的库与 API

库	关键函数	用途
`sounddevice`	`sd.rec(frames, samplerate, channels, dtype)`、`sd.wait()`、`sd.play()`、`sd.query_devices()`	麦克风录音、播放
`soundfile`	`sf.read(path, dtype='float32')`、`sf.write(path, data, fs, subtype='PCM_16')`	高质量 WAV 读写
`scipy.io.wavfile`	`wavfile.read` / `write`	轻量替代方案
`scipy.signal`	`resample_poly(x, up, down)`、`get_window('hamming', N)`、`butter` + `sosfilt`（可选预加重/去直流）	重采样、加窗、滤波
`numpy.lib.stride_tricks`	`sliding_window_view(x, frame_len)[::hop]`	零拷贝分帧（比循环快 50 倍以上）

核心代码骨架

```
# 1) 录音：16 kHz、单声道、5 秒
rec = sd.rec(int(5*fs), samplerate=fs, channels=1, dtype='float32')
sd.wait()
x = rec[:, 0]

# 2) 归一化（只缩不放，避免放大底噪）
x = x / (np.max(np.abs(x)) + 1e-12)

# 3) 分帧：sliding_window_view，零拷贝
frames = np.lib.stride_tricks.sliding_window_view(x, frame_len)[::hop]
frames = frames * np.hamming(frame_len)   # 广播加窗
# frames.shape == (n_frames, 400)
```

最小验证实验

1. 录 3 秒自己的声音，读出采样率，确认是 16000。

2. 用 `resample_poly` 把一个 44.1 kHz 的 WAV 降到 16 kHz，画前后频谱，确认峰值频率没变。
3. 打印 `frames.shape`，手算 `(L-400)//160+1` 验证一致。

常见坑

- `sounddevice` 默认采样率可能被设备强制改掉，**录完必须回读 `rec` 的实际长度并用 `sd.query_devices()` 确认**。
- Windows 下 `sd.rec` 是异步的，忘了 `sd.wait()` 会录到空数据。
- 归一化用 `max|x|` 而非 `sqrt(mean(x²))`，后者会被静音段拉低。
- 不要用 `np.mean` 去直流后再归一化，会破坏击键瞬态。

4.2 板块 B：端点检测与单音分割（成员 2）

要学的理论

- 短时能量本质是**“平方律检波 + 低通滤波”**：先平方（检波），再用滑动窗求和（低通）。这正是课程里“信号通过非线性器件再滤波”的经典结构。
- 短时过零率本质是**“差分 + 取绝对值 + 低通”**：衡量信号在高频段的能量占比。对钢琴而言，击键瞬间含大量高频成分，ZCR 高；音符衰减后 ZCR 降低——**ZCR 是区分“音符起始瞬态”与“稳态衰减”的好特征**。
- 双门限逻辑：高门限确定“一定是音符”，低门限确定“音符的扩展边界”，避免把弱衰减段丢掉。
- 单音时长下限与静音间隔的物理依据：钢琴最快连奏 60–80 ms。

要用的库与 API

库	关键函数	用途
`numpy`	`np.sign`、`np.sum(axis=1)`、`np.percentile`、`np.where`、`np.diff`	向量化计算能量与过零率
`scipy.ndimage`	`median_filter(E, size=5)`	能量曲线平滑，去掉孤立毛刺
`scipy.signal`	`find_peaks(E, height=..., distance=..., prominence=...)`	峰值法找音符中心（辅助手段）
标准库	`csv` / `pandas`	读写标注文件

核心代码骨架

```
E = np.sum(frames**2, axis=1)                                    # 短时能量
Z = 0.5*np.sum(np.abs(np.diff(np.sign(frames), axis=1)), axis=1)  # 短时过零率

E_s = median_filter(E, size=5)
E_noise = np.percentile(E_s[:20], 20)        # 前 200 ms 作为噪声底
T_high, T_low = 5*E_noise, 2*E_noise

# 四状态机：SIL → MAYBE_START → NOTE → MAYBE_END → SIL
```

最小验证实验

1. 对 20 音《玛丽有只小羊羔》，画出能量曲线 + 双门限横线 + 检测到的音符区间（用 `axvspan` 半透明色块），肉眼数一数是不是 20 段。
2. 对 SNR=10 dB 的同一条乐句重复，看漏检/虚检出现在哪里。

3. 与 `labels.csv` 对比，算起点误差的中位数和 95 分位数。

常见坑

- **门限用固定绝对值**（如 `E > 0.01`）→ 换一段录音就全错。必须用噪声底相对门限。
- 音符之间没有静音（连奏）时会被切成一段 → 需要 ZCR 谷点辅助二次分割。
- 钢琴的衰减尾巴很长，终点会被判得太晚 → 终点用低门限 + 最小间隔约束。
- 忘了起点回退（钢琴 attack 上升沿很短，能量过门限时已晚了 10–20 ms）→ 起点统一减 10 ms。

4.3 板块 C：频谱分析与特征提取（成员 3）

要学的理论（这是全组理论含量最高的一节）

1. **DFT 与频率分辨率**：$\Delta f = fs/N$。N=400、fs=16 kHz 时 Δf=40 Hz，而 C4 与 D4 只差 32 Hz——**所以 25 ms 的端点检测帧绝对不能拿来做频谱识别**，必须用整个音符段（≥150 ms）。这是最容易犯的致命错误。
2. **栅栏效应**：DFT 只在 $k \cdot fs/N$ 的离散频点上取样，真实峰值落在两个频点之间时会被低估。补零能加密频点（内插），但不提高真实分辨率。
3. **频谱泄漏**：有限长截断等效于乘矩形窗，频域卷积 sinc 函数。加 Hamming 窗可压旁瓣。
4. **Goertzel = 二阶 IIR 谐振器**：其系统函数为

```
H(z) = (1 - e^{-jω₀}z^{-1}) / (1 - 2cosω₀·z^{-1} + z^{-2})
```

分母的两个极点位于单位圆上 $z = e^{\pm j\omega_0}$，构成一个**无阻尼谐振器**，只对 ω_0 附近的频率响应最大。递推式 $s[n] = x[n] + 2\cos\omega_0 \cdot s[n-1] - s[n-2]$ 就是这个系统的零状态响应（初值 $s[-1]=s[-2]=0$）。最后用

```
|X(ω₀)|² = s[N-1]² + s[N-2]² - 2cosω₀·s[N-1]·s[N-2]
```

还原幅度平方。**这段解释就是答辩时“Goertzel 的系统函数是什么”的标准答案。**

5. **十二平均律**：$f(\text{midi}) = 440 \times 2^{((\text{midi}-69)/12)}$。相邻半音比 $2^{(1/12)} \approx 1.05946$，一个八度频率翻倍。C4=60→261.63 Hz，A4=69→440 Hz，C6=84→1046.50 Hz。

要用的库与 API

库	关键函数	用途
`numpy.fft`	`rfft(x)`、`rfftfreq(N, 1/fs)`、`abs()`	实信号快速傅里叶变换
`scipy.fft`	`rfft`（更快，支持 `workers` 多线程）	大批量计算时用
`scipy.signal`	`find_peaks(spectrum, height=, prominence=, distance=)`、`get_window`、`welch`（可选功率谱估计）	峰值检测、加窗
`numpy`	复数运算、`np.exp`、`np.log2`	频率换算

核心代码骨架

```
# Goertzel：任意频率精确取样，O(N) 每次、O(1) 内存
def goertzel_mag(x, f0, fs):
```

```
    w0 = 2*np.pi*f0/fs
    c  = 2*np.cos(w0)
    s1 = s2 = 0.0
    for xn in x:                    # 可用 numba 加速
        s0 = xn + c*s1 - s2
        s2, s1 = s1, s0
    return np.sqrt(s1*s1 + s2*s2 - c*s1*s2)

# 26 根谐波
f0 = 440 * 2**((midi-69)/12)
K  = min(26, int(np.floor((fs/2) / f0)))      # 高音区越界处理
bins = np.array([goertzel_mag(x, (k+1)*f0, fs) for k in range(K)])
bins = np.pad(bins, (0, 26-K))                # 补零到 26 维
```

最小验证实验

1. 对 C4#（277.18 Hz）画 FFT 幅度谱，确认基频与 2 次谐波（554.36 Hz）都有峰，且 2 次谐波峰值更大——**这是论文图 2 的关键现象**，也是"为什么不能只取基频"的证据。
2. 同一频点上 Goertzel 与 FFT 结果对比，打印相对误差，应 < 1e-9。
3. 用 25 ms 帧做一次 C4/D4 的频谱，观察两者混叠无法区分——**亲手验证 4.3 节第 1 条**，这个实验做一遍，比看十遍书都记得住。

常见坑

- Goertzel 的 `c = 2cos(ω₀)` 里的 ω₀ 用 `2πf₀/fs`，不是 `2πk/N`。**Goertzel 可以取任意频率，这正是它相对 FFT 的优势。**
- 幅度开方后忘记做归一化，导致不同音量下模板不匹配。
- `find_peaks` 的 `height` 用绝对值，遇到录音音量不同就失效 → 改用相对最大值或 `prominence`。
- 谐波数固定取 26，高音区（C6 的 8 次谐波已超 8 kHz）会拿到一堆 0，归一化后维度退化。

4.4 板块 D：模板库与匹配滤波识别（成员 4）

要学的理论

- **匹配滤波器的定义**：在白噪声背景下，使输出信噪比最大的线性滤波器，其冲激响应为 h(t)=s(T−t)，即**信号的时间反转平移**。频域上 H(ω)=S*(ω)。
- **与相关的等价性**：y(t)=s_i(t)∗h_j(t)，在 t=T 采样得 ∫s_i(τ)s_j(τ)dτ = 相关系数。推导见 2.6 节。
- **Cauchy–Schwarz 保证最优性**：等能量条件下，自相关恒 ≥ 互相关，故 argmax 判决是最优的。
- **为什么保留泛音能抗噪**：基频只是 26 维中的 1 维，噪声对单根谱线的影响是随机的；用 26 维做内积相当于 **26 次独立观测的平均**，方差降为 1/26。这是论文"利用泛音提高抗噪性能"的定量解释，报告里写上这一句，档次立刻不一样。
- **五度/八度混淆的成因**：C4=261.63、G4=392.00，G4 ≈ 1.5×C4，即 G4 是 C4 的第 3 次谐波；C5 = 2×C4。所以 C4 的模板与 G4、C5 的模板天然有非零相关（论文表 2 中 C4–G4 为 0.107 正好印证）。

要用的库与 API

库	关键函数	用途
`numpy`	`@` 矩阵乘、`argsort`、`argmax`、`np.linalg.norm`	一次算完 26 路匹配
`scipy.optimize`	`nnls(A, b)`	改进版和弦分解

库	关键函数	用途
`fluidsynth`（命令行）+ 声卡	`fluidsynth -ni FluidR3_GM.sf2 in.mid -F out.wav -r 16000`	批量合成 26 个标准音
`pretty_midi` / `mido`	生成单音 MIDI 供合成	模板制作

核心代码骨架

```
# 模板矩阵 T: (26, N)，每行是能量归一化后的标准音频谱
T = np.array([normalize(t) for t in templates])        # ‖T[j]‖₂ = 1

# 26 路匹配滤波 = 一次矩阵乘法（等价于 26 次卷积在 n=N-1 处的采样）
scores = T @ s_i          # s_i 也已 L2 归一化，scores[j] = r_ij[0] ∈ [-1, 1]

j = int(np.argmax(scores))
accepted = scores[j] > tau and (scores[j] - np.sort(scores)[-2]) > margin
note = NOTE_NAMES[j]
```

最小验证实验

1. 复现论文表 2：对 7 个白键的干净录音算 7×7 相关矩阵，用 `imshow` 画热图，对角线应在 0.85 以上。
2. 打印 C4 与 G4 的相关系数，确认在 0.1 量级，并在报告里解释成因。
3. 单元测试：验证 `scores` 恒 ≤ 1.0（Cauchy–Schwarz 上界），且自相关 = 1.0（浮点误差内）。

常见坑

- 模板和待测信号**归一化方式不一致**（一个用 max，一个用 L2）→ 相似度全部失真。必须都做 L2 归一化。
- 写 26 次 `np.convolve` 循环 → 慢 30 倍，且代码难看。用矩阵乘法。
- 只取 argmax 不设阈值 → 静音段也会被强行判成一个音。必须加 `accepted` 拒识。
- 模板是合成音、测试是真实钢琴，音色差异导致相关系数整体下降到 0.7 左右但仍能正确排序——**这说明排序比绝对值更重要**，报告里要点出来。

4.5 板块 E：和弦分解与乐句识别（成员 4、成员 5）

要学的理论

- 和弦的三个音（C4-E4-G4）频谱叠加后，**各音的谐波列互相交叠**：C4 的 3 次谐波 784.9 Hz 与 G4 的 2 次谐波 784.0 Hz 几乎重合。这导致“相关度”不再等于“该音在和弦中的能量占比”——**这正是论文表 3 中 E4(0.706) > C4(0.569) 的原因，也是 Top-K 方法的原理缺陷。**
- 改进思路：把和弦频谱看作若干单音模板的**非负线性组合** $b \approx A \cdot w$，$w \geq 0$，用 NNLS 求稀疏解，w 非零的位置就是和弦的音。
- 乐句识别的本质是**序列对齐**：识别序列与标准乐谱序列做编辑距离（插入/删除/替换），准确率 = 1 − 编辑距离/标准长度。

要用的库与 API

库	关键函数	用途
`scipy.optimize`	`nnls`	改进版和弦分解
`difflib`	`SequenceMatcher(None, a, b).ratio()`	编辑距离/相似度

库	关键函数	用途
mido	MidiFile、MidiTrack、Message('note_on'/'note_off')	MIDI 导出
pretty_midi	PrettyMIDI、Instrument、write()	更易用的 MIDI 导出
music21（可选）	stream、note.Note、write('musicxml')	生成五线谱（加分项）

最小验证实验

1. 复现论文表 3：三音和弦的 26 维相关度排序，Top-3 是否命中。
2. 用 NNLS 求同一和弦的权重 w，对比两者的音名集合。
3. 导出《玛丽有只小羊羔》的 MIDI，用 Windows 播放器打开听一遍——**听感对了，说明整条链路通了**。

常见坑

- MIDI 的 note_on 必须配对 note_off，否则音符永远不结束。
- MIDI 音高号用 60 表示 C4（不是 0 或 1），与我们的 midi 字段定义一致，别混。
- 导出时用了识别错误的音，听感会很怪——先确保识别对了再导出。

4.6 板块 F：性能评估与抗噪实验（成员 2、成员 5）

要学的理论

- **SNR 定义**：SNR_dB = 10·log₁₀(P_signal / P_noise)，其中 P = mean(x²)。反解噪声标准差：σ = sqrt(P_signal / 10^(SNR/10))。
- **为什么频域抗噪**：白噪声功率均匀分布在整个 0–8 kHz，而乐音能量集中在 26 根离散谱线上。噪声落在谱线上的比例约为 26×(谱线有效宽度)/带宽，很小。**“时域波形完全失真但频谱仍可用”就是这个原因**——这句话要写进报告，是论文的核心论点。
- **评估的严谨性**：固定随机种子、固定测试集、每档 200 次检测、报告置信区间或标准差。

要用的库与 API

库	关键函数	用途
numpy.random	default_rng(2026)、.normal(0, sigma, size)	可复现加噪
pandas	DataFrame.to_csv	实验数据归档
matplotlib	errorbar、imshow（混淆矩阵）、bar	出图
sklearn.metrics（可选）	confusion_matrix	混淆矩阵（也可手写 26×26 矩阵）
time	perf_counter()	耗时基准

核心代码骨架

```
rng = np.random.default_rng(2026)
Px  = np.mean(x**2)
for snr_db in [20, 15, 10, 5, 0]:
    sigma = np.sqrt(Px / 10**(snr_db/10))
    n = rng.normal(0, sigma, size=x.shape)
    y = x + n            # 加性高斯白噪声
```

最小验证实验

1. 画 SNR=10 dB、0 dB 下的波形对比图 + 频谱对比图，**用图直接说明"时域看不出、频域还能用"**。
2. 确认 5 档 × 10 轮 × 20 音符 = 1000 次检测跑完的耗时（应 < 10 分钟，否则需要优化）。
3. 两次相同种子跑出的结果完全一致。

常见坑

- 噪声加在**归一化之后**，否则 Px 计算错误。
- 每轮都要重新生成噪声，不要用同一段噪声跑 10 次（否则统计无效）。
- 错检率的分母是"检测总次数"，不是"音符数"，别算错。

4.7 板块 G：图形界面与工程化（成员 5）

要学的理论（这块偏工程，但答辩会问"为什么界面能实时"）

- 生产者–消费者模型：录音线程（生产者）→ 缓冲区 → 识别线程/主线程（消费者）。界面卡死的根本原因是把耗时计算放在了主线程。
- 帧率与延迟：GUI 刷新 20–30 fps 即可，识别一帧应在 50 ms 内完成。

要用的库与 API

方案一：PySide6（推荐，专业、可打包）

模块	关键类/函数
主窗口	`QMainWindow`、`QWidget`、`QVBoxLayout`、`QGridLayout`
控件	`QPushButton`、`QLabel`、`QComboBox`、`QSlider`、`QFileDialog`、`QTableWidget`
绘图嵌入	`matplotlib.backends.backend_qtagg.FigureCanvasQTAgg`、`NavigationToolbar2QT`
多线程	`QThread` + `Signal/Slot`（**录音必须放这里**）
定时器	`QTimer` 刷新波形

方案二：Streamlit（最快出原型，2 天可完成，作为 Plan B）

功能	API
麦克风录音	`st.audio_input("点击录音")`（需较新版本）
文件上传	`st.file_uploader`
画图	`st.pyplot(fig)`
侧边参数栏	`st.sidebar.slider("帧长/ms", 10, 50, 25)`
状态展示	`st.metric("识别结果", "C4")`、`st.dataframe`
启动	`streamlit run app.py`

最小验证实验：做一个只有"录音按钮 + 显示时长"的界面，点击后 3 秒内看到时长。**这一个 demo 能在半天内完成，先跑通它，再加功能。**

常见坑

- 在 `QThread` 里直接操作界面控件 → 崩溃。必须用信号槽。
- matplotlib 中文在打包后丢失字体 → 用 `font_manager.fontManager.addfont()` 指定字体文件路径。

- PyInstaller 打包后 `sounddevice` 找不到 `_sounddevice_data/portaudio-binaries` → 加 `--add-data`，或用 `--collect-all sounddevice`。
- 用 `--onefile` 启动慢（5–10 秒解压）→ 现场演示用 `--onedir`。

4.8 板块 H：报告与答辩（全员，成员 1 统稿）

报告结构与分工

章节	内容	主笔	对应评分
封面 + 摘要	题目、组号、5 人分工与工作量、300 字中文摘要 + 英文摘要	成员 1	—
1 引言	研究背景（音乐检索/自动谱曲）、文献综述、本文工作	成员 1	10%
2 案例阐述及方案设计	2.1 乐音特性与十二平均律、2.2 端点检测原理、2.3 频谱分析、2.4 匹配滤波与相关、2.5 Goertzel、2.6 系统框图与流程	成员 3 主笔，成员 2、4 补各自小节	**40%**
3 实验结果与分析	3.1 实验环境与数据集、3.2 端点检测结果、3.3 单音识别与 7×7 相关矩阵、3.4 和弦分解、3.5 乐句识别、3.6 抗噪实验、3.7 消融对比、3.8 性能与复杂度、3.9 与论文结果对比	成员 2、4、5 分写，成员 1 统	**40%**
4 结论	工作总结、局限、改进方向	成员 1	10%
参考文献	≥ 8 篇，GB/T 7714 格式	成员 1	—
附录	核心代码、使用说明、分工表	成员 1	—

格式硬性要求（直接照抄，逐条打勾检查）

项目	要求
纸张	A4，通栏排版
页边距	上 2.2 cm、下 2.0 cm、左 2.7 cm、右 2.3 cm
页码	居中，必须插入
行距	摘要与参考文献单倍行距，其余一律 1.2 倍
字体	中文宋体，英文与数字 Times New Roman
字号	摘要与文献小五号，正文五号；一级标题小四粗体（段前 0.5 行），二级标题五号粗体
缩进	每段首行缩进约 2 字符，段前段后不留空行
图表	图题在下、表题在上，编号连续（图 1、图 2……），图内文字 ≥ 8 pt

答辩分工（每人对自己的板块负责，不许替答）

环节	时长	主讲	内容
开场与背景	1 min	成员 1	选题理由、复现目标
原理讲解	3 min	成员 3	十二平均律、Goertzel、匹配滤波与相关
现场演示	3 min	成员 5	麦克风录一句乐句 → 出识别结果 → 导出 MIDI
实验结果	3 min	成员 2、4	相关矩阵、抗噪曲线、消融对比
提问	5–8 min	全员	各自回答自己模块的问题（见 7.2 节）

---

5 七周排期与阶段验收

5.1 总甘特图

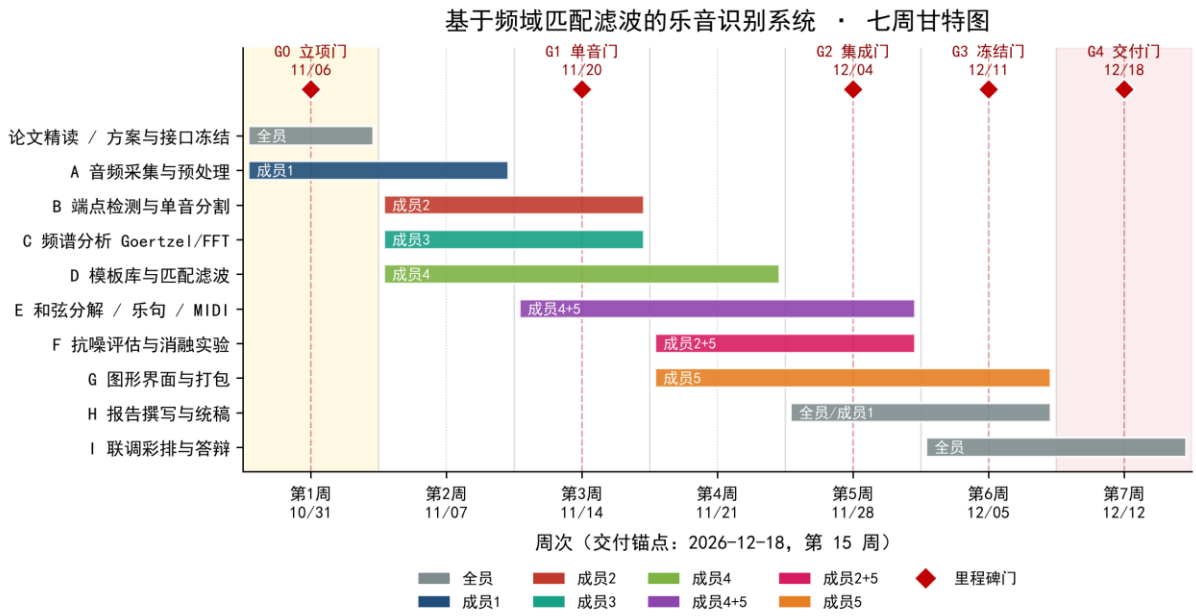


图 5-1 项目甘特图

> 图中每行是一个工作包，颜色代表负责人，菱形是里程碑门。**新增任务一律先画到图上再开工**，画不进去就说明排不下。

5.2 五道里程碑门（不达标不进下一阶段）

每道门都有一张**必须全部打勾**的检查表。任何一项不通过，当周周末加一次 2 小时集中攻坚，不允许“先往下走，回头再补”。

G0 立项门 · 11 月 6 日（周五）22:00

检查项	判定方式
方案一页图纸（系统框图 + 六步流程）	群里发图，全员确认
`types.py` 五个数据结构定稿并打 tag `v0.1-contract`	Git 上有 tag
`config.py` 参数表全部落成常量	代码可 import
26 个标准音模板 WAV 已生成、能读进来	跑 `demo_audio.py` 打印 26 个时长
录音 demo 能录 3 秒、采样率确认 16000	现场演示
论文精读笔记（每人 1 页，写清自己负责的那一步）	群文件 5 份
验收日期已向老师确认	群公告写明

G1 单音门 · 11 月 20 日（周五）22:00

检查项	判定方式
单音端到端识别准确率 ≥ 90%（26 音 × 3 次干净录音）	`evaluate.py --mode single` 输出报告
7×7 相关矩阵复现，对角线 ≥ 0.85、非对角 ≤ 0.20	热图截图
C4# 的 FFT 谱与 Goertzel 离散谱线对照图完成	与论文图 2/图 4 并排对比
Goertzel 与 FFT 交叉验证相对误差 < 1e-9	单元测试通过
端点检测在干净乐句上 F1 ≥ 0.95	`labels.csv` 对比
每个模块都有可独立运行的 `demo_xxx.py`	5 个 demo 全部能跑

G2 集成门 · 12 月 4 日（周五）22:00

检查项	判定方式
20 音乐句识别准确率 ≥ 85%	命令行跑 3 条乐句
GUI 能录音、能显示波形/频谱/音名	现场演示
三音和弦 Top-3 命中率 ≥ 80%	5 个和弦样本
抗噪实验 5 档 × 200 次跑完，错检率–SNR 曲线出图	`figures/snr_curve.png`
消融实验（仅基频 vs 全模板）有数据	对比表
130 个单音样本 + 3 条乐句 + 5 个和弦数据集完整、可重建	`make_dataset.py` 一键生成

G3 冻结门 · 12 月 11 日（周五）22:00

检查项	判定方式
代码冻结：只修 bug，不加功能	Git 打 tag `v1.0-rc`
报告初稿完整（四部分齐全，图表 ≥ 10 张）	发 PDF 给全组
格式检查清单全部通过（附录 B）	逐条打勾签字
可执行文件打包成功或在第二台电脑上跑通	现场验证
README 能让一个外人 10 分钟内跑起来	找一位非本组同学实测
答辩 PPT 初稿完成	12–15 页

G4 交付门 · 12 月 18 日

检查项	判定方式
彩排 ≥ 2 次，全流程 ≤ 12 分钟	计时
现场演示一次成功（含麦克风实录）	彩排即演练
报告终稿 + 代码 + 可执行文件按“组号”命名打包提交	检查文件夹名
演示视频备份 1 段（应对现场设备故障）	存在本地和云端

5.3 逐周任务表（含周末验收标准）

周次	日期	全员共同动作	各板块本周内容	周末验收标准（做到才算过关）
W1	10/31–11/06	① 开 60 分钟启动会（见 6.4）；② 各自精读论文并写 1 页笔记；③ 装环境、跑通 4.0 节最小实验	A：`types.py`、`config.py`、录音+分帧；C：十二平均律频率表 + FFT demo；D：装 `fluidsynth`、合成 26 个模板；B/G：读论文第 2 章、调研工具	**G0 门全部打勾**：5 人 `test_440hz.png` 都能生成且中文正常
W2	11/07–11/13	站会 15 min × 2；周日 30 min 周会	A：重采样、加窗、单元测试；B：`vad.py` 初版（能量+过零率+双门限）；C：Goertzel 实现 + 与 FFT 交叉验证；D：模板加载器 + L2 归一化；E/G：PySide6 装好，跑通“Hello Window”	三条独立曲线能画出来：① 波形+分帧能量 ② C4# 的 FFT 谱 ③ Goertzel vs FFT 误差图
W3	11/14–11/20	① 联调会 1 次（2 小时）；② 把 `main_demo.py` 串起来	A：集成主流程；B：门限标定 + 人工标注 `labels.csv`；C：谐波拾取 + 26 维特征向量输出；D：26 路匹配滤波 + 判决 + 拒识	**G1 门**：单音 ≥ 90%、7×7 矩阵复现、5 个 demo 可跑、Goertzel 误差 < 1e-9
W4	11/21–11/27	站会；周会；开始写报告第 2 章	B：数据集补齐 130 单音 + 3 乐句 + 5 和弦；C：报告 2.1/2.4/2.5 初稿；D：26 音全量测试 + `chord.py`；E：`melody.py` 串接 + 编辑距离；G：GUI 骨架（录音按钮 + 波形显示）	命令行能对 20 音乐句输出完整音名序列；GUI 能录音并画出波形；和弦 Top-3 能跑通

周次	日期	全员共同动作	各板块本周内容	周末验收标准（做到才算过关）
W5	11/28–12/04	① 联调会 2 次；② 报告 3 章素材汇总	B+F：抗噪实验 5 档 × 200 次；C：理论串讲 40 min（全组必到）；E：MIDI 导出 + 批量测试；G：GUI 完整化（四区布局、参数面板、导出按钮）	**G2 门**：乐句 ≥ 85%、GUI 现场录音出结果、抗噪曲线出图、消融有数据
W6	12/05–12/11	① 报告统稿（成员 1 主导，2 轮）；② 格式自检	全员：补实验、补图、写各自章节；G：打包 exe / `run.bat`；H：PPT 初稿；1：术语与编号统一	**G3 门**：代码冻结 tag、报告初稿完整、打包成功、README 通过外人实测
W7	12/12–12/18	① 彩排 2 次（12/14、12/16）；② 提交；③ 现场答辩	全员：只修 bug、只改表述；5：演示流程固化；3：理论问答演练；1：材料打包与提交	**G4 门**：彩排通过、材料提交、演示一次成功

5.4 各板块 × 各阶段的内容与验收标准（核心表）

板块	前期（W1–W2）要做出什么	中期（W3–W4）要做出什么	后期（W5–W6）要做出什么	阶段验收标准
0 管理	启动会、`types.py`/`config.py` 冻结、看板建好、验收日期确认	每周站会+周会、G1 联调、风险登记册更新	报告统稿 2 轮、G3 冻结、提交材料清单	会议记录 7 份、周报 7 份、tag ≥ 3 个、无“卡住超 3 天无人管”的任务
A 采集预处理	录音/读文件/重采样/归一化/分帧/加窗 + 单元测试	集成进 `main_demo.py`，支持批量读文件夹	修边界 bug（超短音频、全零、削顶）	16 kHz 单声道可读；归一化后 max\
B 端点检测	`vad.py` 初版，能量/过零率曲线可画出	门限标定、130 样本标注完成、F1 ≥ 0.95	SNR 10 dB 下 F1 ≥ 0.85；抗噪实验执行完毕	干净乐句 F1 ≥ 0.95；起点误差中位数 ≤ 30 ms、95 分位 ≤ 80 ms；5 s 音频处理 ≤ 0.5 s
C 频谱分析	频率表、FFT 频谱、中文标注图	Goertzel 实现并与 FFT 交叉验证；输出 26 维特征向量	高音区越界处理；报告 2 章定稿；理论串讲	Goertzel vs FFT 相对误差 < 1e-9；C4# 谱形与论文图 2 一致；26 音 × 26 谐波计算 ≤ 20 ms
D 匹配滤波	26 个模板合成、加载、L2 归一化	26 路匹配 + 判决 + 拒识；复现表 2	26 音全量测试、混淆矩阵、消融实验	7×7 对角 ≥ 0.85、非对角 ≤ 0.20；单音 ≥ 95%；26 路匹配 ≤ 1 ms/帧
E 应用层	无（依赖 B、D）	和弦 Top-3 复现表 3；乐句串接 + 编辑距离	NNLS 改进对比；MIDI 导出可播放	和弦 Top-3 ≥ 90%；20 音乐句 ≥ 90%；MIDI 可被 MuseScore 正确打开
F 评估	无	数据集与标注就绪；评估脚本框架	5 档抗噪跑完；3 张图完成	SNR 20/15/10 dB 错检 ≤ 2%，5 dB ≤ 5%，0 dB ≤ 10%；固定种子可复现；每档 ≥ 200 次
G 界面	环境调研、Hello Window	GUI 骨架：录音按钮 + 波形显示	四区布局 + 参数面板 + 导出；打包	启动 ≤ 5 s；点录音到出结果 ≤ 1 s；连续 10 次不崩溃；第二台电脑可运行
H 报告	精读笔记、引言素材	第 2 章初稿	第 3 章 + 第 4 章 + 格式统一 + 定稿	四部分齐全、图表 ≥ 10 张、格式清单全过、查重 < 20%

5.5 量化验收标准总表（这张表决定我们拿 A 还是 B）

编号	指标	目标值	测量方法	责任人	门

编号	指标	目标值	测量方法	责任人	门
P1	单音识别准确率（干净录音，26 音 × 5 次 = 130）	**≥ 95%**	`evaluate.py --mode single`	成员 4	G1/G3
P2	7×7 相关矩阵对角线	≥ 0.85	与论文表 2 对比	成员 4	G1
P3	7×7 相关矩阵非对角	≤ 0.20	同上	成员 4	G1
P4	端点检测 F1（干净乐句）	≥ 0.95	与 `labels.csv` 对比	成员 2	G1
P5	端点检测起点误差中位数 / 95 分位	≤ 30 ms / ≤ 80 ms	同上	成员 2	G1
P6	Goertzel 与 FFT 一致性（相对误差）	< 1e-9	单元测试	成员 3	G1
P7	26 音乐句识别准确率（≥ 20 音符，3 条）	**≥ 90%**	与标准乐谱编辑距离	成员 5	G2/G3
P8	三音和弦 Top-3 命中率	≥ 90%	5 个和弦样本	成员 4	G2
P9	错检率 @ SNR 20 / 15 / 10 dB	≤ 2%	每档 200 次	成员 2	G2
P10	错检率 @ SNR 5 dB	≤ 5%	同上	成员 2	G2
P11	错检率 @ SNR 0 dB	≤ 10%	同上	成员 2	G2
P12	消融：0 dB 下“全模板”相对“仅基频”的准确率提升	≥ 10 个百分点	对照实验	成员 2	G2
P13	单帧识别耗时（26 路匹配）	≤ 1 ms	`perf_counter`	成员 4	G2
P14	5 秒音频完整处理耗时	≤ 0.5 s	同上	成员 1	G2
P15	GUI 启动时间	≤ 5 s	秒表	成员 5	G3
P16	GUI 点“录音”到出结果	≤ 1 s（3 秒录音）	秒表	成员 5	G3
P17	现场演示连续成功次数	≥ 10 次不崩溃	现场	成员 5	G4
P18	报告字数 / 图表数	≥ 8000 字 / ≥ 10 张	统计	成员 1	G3
P19	报告查重率	< 20%	自查工具	成员 1	G3
P20	答辩全流程时长	≤ 12 min	彩排计时	成员 1	G4

> **关于目标值的说明**：论文在 SNR=0 dB 时错检率 4.5%，我们定 10%，是因为我们的录音设备（手机/笔记本麦克风）和环境噪声远不如论文的实验条件，定 4.5% 是不现实的。**报告里必须写清这个差距的原因**——“承认差距并解释原因”比“虚报一个好看的数”更能得分，老师一问就露馅的风险也小得多。

5.6 怎么测才可信（评估纪律）

1. **测试集与模板必须分离**。模板用合成音，测试用真实录音——否则等于自己考自己。
2. **先冻结测试集，再调参**。`labels.csv` 和 130 个样本在第 5 周后不再新增，否则前后数据不可比。
3. **每次实验记录三样东西**：随机种子、代码 commit 号、参数配置。存成 `results/日期_实验名.csv`。
4. **报平均值的同时报样本量**。写“95%（130/130 样本）”而不是写“95%”。
5. **不确定的样本单独统计**。被 `margin` 拒识的样本要单独报数量，不能悄悄算成错误或正确。
6. **至少一次“陌生人测试”**。找一个没参与项目的同学，给他 README 和麦克风，看他能不能在 10 分钟内跑出结果——这是“工程规范”这一项最真实的检验。

---

6 五人高效合作建议

> 这一章不是通用鸡汤，每一条都对应本项目会真实发生的一次事故。

6.1 三条铁律（违反任何一条，这个项目大概率翻车）

铁律一：任何模块都必须有一个“唯一负责人”，且只有一个人。

“我们一起做端点检测”= 没有人做端点检测。正确说法是：“成员 2 负责 `vad.py` 的交付，成员 3 帮忙 review 算法，但代码只有成员 2 能合入。”

铁律二：接口先冻结，代码后开工。

第 1 周宁可少写 200 行代码，也要把 `types.py` 定下来。本项目的典型崩盘场景是：成员 2 的 `NoteSegment` 用“样本索引”、成员 3 以为是“秒”，第 5 周集成时 20 个音符全部错位，返工三天。**这类事故 100% 可以在第 1 周用一页纸避免。**

铁律三：第 3 周必须跑通一次完整闭环。

哪怕准确率只有 60%、界面还是命令行、频谱图丑得不能看，也要有一次“录音 → 切音 → 特征 → 判决 → 打印音名”的端到端跑通。**闭环越早，五个人的方向越不会偏；闭环越晚，返工越贵。**

6.2 接口先行：具体怎么落地

1. **第 1 周周三前**，成员 1 把 `types.py` 写成可运行的 `@dataclass`，每个字段带中文注释和单位说明，例如：

```
@dataclass
class NoteSegment:
    t_start: float      # 起点，单位：秒（相对录音开头）
    t_end: float        # 终点，单位：秒
    samples: np.ndarray # 该段的原始归一化波形，float32
    index: int          # 在整段录音中的序号，从 0 开始
```

2. **第 1 周周五评审会**，逐字段过一遍，重点确认三件事：单位、取值范围、空值怎么表示。评审通过后 `git tag v0.1-contract`。
3. **之后任何改动必须走流程**：提 Issue → 说明影响哪几个人 → 全组知情 → 改完通知下游 → 更新 tag。**口头说一声就改接口，是这个项目最贵的错误。**
4. **每个模块配一个** `demo_xxx.py`，用假数据或一小段真实数据独立跑通。这样成员 4 不必等成员 2 的数据集做完就能开发。

6.3 Git 工作流与文件规范

项目	约定
仓库	Gitee 或 GitHub 私有仓库，5 人全部有写权限
分支	`main`（保护，只接受 PR）＋ 每人一条 `feat/模块名`，如 `feat/vad`
合并	至少 1 人 review 后合并；**禁止直接 push main**
提交信息	`[模块] 动词 + 内容`，例：`[vad] 增加最小音长约束`
打标签	`v0.1-contract`（W1）、`v0.5-single`（W3）、`v0.8-integration`（W5）、`v1.0-rc`（W6）、`v1.0-final`（W7）
大文件	`data/`、`*.wav`、`*.sf2` 加入 `.gitignore`；用 `make_dataset.py` 一键重建，**不要把音频塞进 Git**
依赖	`requirements.txt` 锁版本；任何新增依赖先在群里说一声
命名	代码 `snake_case`，类 `PascalCase`，常量 `UPPER_CASE`；文件 `模块_功能.py`

`.gitignore` 建议内容：

```
__pycache__/
```

```
*.pyc
data/*.wav
data/*.sf2
dist/
build/
*.spec
results/*.csv
report/*.pdf
```

6.4 会议机制（不占时间，但必须开）

会议	频次	时长	议程	产出
站会	每周 2 次（周二、周四晚）	15 min	每人只答三问：上次做了什么 / 下次做什么 / 卡在哪	群消息里三条待办
周会	每周日 20:00	30–40 min	① 本周验收标准是否达成 ② 演示一个能跑的东西 ③ 下周任务确认 ④ 风险与求助	会议记录（谁/做什么/何时/交付物）
联调会	W3、W5 各 1 次（必要时加）	2 h	现场合并代码、跑通全流程、修接口问题	可运行的 `main_demo.py`
理论串讲	W5 一次	40 min	成员 3 讲透全链路数学	每人能讲自己的三行

站会的三条纪律：① 站着开、限时 15 分钟；② 只讲结论不讲过程；③ 需要深入讨论的问题会后再拉人，**绝不在站会上展开技术辩论**。

6.5 结对与知识备份（专门针对“能力有限”）

大二学生的能力差异是客观存在的，解决办法不是“强的多干”，而是**结对**：

结对	组合理由	具体做法
成员 1 ↔ 成员 5	都在系统层，接口最密集	成员 5 写 GUI 调成员 1 的 API；成员 1 帮成员 5 排查打包问题
成员 2 ↔ 成员 3	都在信号处理层	成员 3 帮成员 2 用向量化改写循环；成员 2 帮成员 3 验证频谱结果
成员 4 + 成员 1 兜底	识别核心最重	成员 1 每周看一次 `matcher.py`，确保能接手

具体规则：

- **“备份人”制度**：每个人的核心模块，必须有一个能看懂、能改的备份人。备份人不需要写代码，但要能回答“这段在干什么”。
- **代码走读**：每周周会上，随机抽一个人讲别人上周写的 20 行代码。讲不出来说明注释不够或没备份，当场补。
- **禁止“黑盒交付”**：任何代码合入前，作者要能对着屏幕讲清每一步。讲不清的代码一律退回——**这不是刁难，是防止第 7 周他请假全组崩盘。**
- 如果第 2 周发现某人确实跟不上：**降低难度而不是加人**。例如把“实现 Goertzel”换成“用 FFT 实现、并验证与论文图一致”，把“做 GUI”换成“做 Streamlit 原型”。**保住每个人的产出，比保住任务的技术含量重要。**

6.6 分歧怎么决策、进度怎么兜

技术分歧（24 小时决策规则）

1. 提方案的人必须同时说清**成本和风险**，不能只说“这样更好”。
2. 相关模块负责人必须表态。

3. 组长 24 小时内拍板，并在群里记录理由。**允许有人不同意，但不允许有人不执行。**

4. 涉及“老师会不会认可”的问题，带两个方案去问老师，不要空手问。

进度失控的三级处置

信号	处置
任务“进行中”超过 3 天没变化	组长私聊问一句：是任务太大、卡住了、还是时间冲突？
某人在 G1/G2 门上连续两次未达标	降级任务范围（砍功能保交付），同时启动结对支援
第 6 周仍有 P0 功能未完成	**立即砍 P1/P2，全员转向保 P0**。宁可少一个加分项，不可少一个核心功能

6.7 风险登记册（第 1 周就建，每周更新状态）

编号	风险	概率	影响	预案	触发条件
R1	麦克风质量差/环境噪声大，现场录音识别率低	高	高	准备“文件输入”备用路径；现场用手机播放预录音频作为兜底	G2 现场演示失败 ≥ 2 次
R2	模板音色与测试音色不匹配，准确率上不去	中	高	三套模板（合成/采样/实录）可切换；必要时用测试集的一个子集做模板（并在报告注明）	G1 单音 < 85%
R3	端点检测调参陷入死循环	中	中	G1 后冻结门限，只允许改后处理	W3 结束仍未达标
R4	GUI 打包失败	中	中	退回“源码 + run.bat + 环境说明”；再不行用 Streamlit	W5 首次打包失败
R5	某成员中途掉线（生病/课程冲突）	中	高	备份人制度；关键模块不留单人知识	连续 5 天无提交
R6	期末考试周与项目冲刺撞车	高	高	第 6 周前完成 90% 工作量，W7 只做彩排和提交	—
R7	报告查重不过	低	高	公式自己推、图自己画、不整段引用译文；W5 自查一次	自查 > 20%
R8	老师现场提问答不上来	中	中	第 7 章 10 个必答题演练；每人只管自己模块	彩排被问倒

6.8 让贡献可见（工作量比例怎么定才服人）

1. **周报里记录每人完成项**，7 份周报就是最终工作量比例的原始依据。
2. **Git 提交记录只作参考，不作唯一依据**——写报告、做数据集、调参同样是工作。建议按“任务卡完成数 × 难度系数”计分。
3. **第 6 周做一次工作量核定会**，每人先自评，再互评，组长汇总。有争议当场对账（看周报和任务卡）。
4. **把分工写进报告封面页和附录**，课程模板本来就要求填“工作量比例”。**公开透明是最好的防划水手段。**
5. 帮忙联调、补文档、做数据的人要在组内公开感谢——小团队最怕“干多干少一个样”。

6.9 用 AI 辅助的边界（大二学生必须提前说清的事）

可以用：查 API 用法、解释报错、生成样板代码、润色文字、翻译英文摘要、帮忙想测试用例。

不可以：

1. **不可以让 AI 写你不理解的算法**。答辩时老师问“这段为什么这么写”，答不上来就是 D 档“知其然而不知其所以然”。

2. **不可以直接粘贴 AI 生成的报告段落**。查重和文风都会暴露，且这门课明确要求“独立撰写，不可简单剪贴现成资料”。
3. **不可以让 AI 编造实验数据**。数据必须是自己跑出来的，这是诚信声明的底线。

组内规则：任何 AI 辅助生成的代码合入前，作者必须能逐行讲清，并在提交信息里注明“AI 辅助”。**这条写进 README，全组签字。**

---

7 答辩准备

7.1 90 秒现场演示脚本（背下来）

时间	动作	台词
0–10 s	打开程序	“我们的项目复现了 2018 年《实验科学与技术》上一篇基于频域匹配滤波的乐音识别论文。”
10–25 s	点“录音”，弹 5 个音（C4 D4 E4 F4 G4）	“现在我对着麦克风弹五个音，采样率 16 kHz。”
25–45 s	界面自动出现波形分段色块 + 频谱 + 大字音名	“系统先用短时能量和过零率把音符切开，再用 Goertzel 取 26 根谐波谱线。”
45–65 s	点“显示匹配度”	“这是 26 路匹配滤波的输出，正确音符的相关度是 0.97，第二名只有 0.10。”
65–80 s	点“导出 MIDI”，用播放器播放	“识别结果直接生成 MIDI，可以播放——这对应论文里的自动谱曲应用。”
80–90 s	切到抗噪曲线图	“在 0 dB 噪声下我们的错检率是 8%，论文是 4.5%，差距主要来自录音设备。”

> **主动说出差距，比被问出来强十倍。** 老师最反感的不是“做得不够好”，而是“夸大成果”。

7.2 十个必答题（每人对自己模块的题必须能脱稿回答）

#	问题	答题要点	谁答
1	为什么用匹配滤波，不直接找基频峰值？	① 只取基频只用 1 维信息，噪声下易错；② 26 维内积相当于 26 次独立观测平均，方差降为 1/26；③ 保留泛音能利用音色特征，抗噪性更强	成员 4
2	匹配滤波器和相关是什么关系？	h[n]=S_j[N−1−n] 时，y[N−1]=ΣS_i[k]S_j[k]=内积；写出 2.6 节推导	成员 4
3	Goertzel 比 FFT 快吗？	**不快**。K 根谱线代价 O(KN)，FFT 是 O(N log₂N)。真正的优势是：可在任意频率精确取样、无需补零对齐、O(1) 内存、纯实数递推易定点化	成员 3
4	Goertzel 的系统函数是什么？	H(z)=(1−e^{−jω₀}z^{−1})/(1−2cosω₀z^{−1}+z^{−2})，极点在单位圆上，是无阻尼二阶谐振器	成员 3
5	为什么频谱要做能量归一化？	使 ‖S‖₂=1 后，由 ΣS_i²−ΣS_iS_j=½Σ(S_i−S_j)²≥0 可得自相关恒 ≥ 互相关，argmax 判决最优；同时消除音量影响	成员 4
6	为什么 C4 和 G4 的相关度有 0.1？	纯五度频率比 3:2，G4 恰是 C4 的第 3 次谐波，两者谐波列部分重合；C5 更极端（2 倍频），所以八度最容易混	成员 3
7	为什么端点检测要同时用能量和过零率？	能量反映响度（平方律检波+低通），过零率反映高频成分（差分+绝对值+低通）；击键瞬态能量低但 ZCR 高，单用能量会漏掉起音	成员 2
8	采样率为什么选 16 kHz？	识别上限约 8 kHz，Nyquist 要求 fs>16 kHz 取等号；44.1 kHz 算力是它的 2.75 倍而无信息增益	成员 3

#	问题	答题要点	谁答
9	模板是怎么来的？换成真实钢琴还准吗？	SoundFont 合成 + 真实采样 + 现场录音三套；实测音色变化会让绝对相关度从 0.97 降到 0.7 左右，但排序不变，故仍可正确识别——这正是匹配滤波的鲁棒性来源	成员 4
10	和弦分解为什么 E4 的相关度最高？	三个音的谐波列交叠，相关度不再等于能量占比；这是 Top-K 方法的原理缺陷，所以我们引入 NNLS 稀疏分解	成员 4 / 成员 5

补充追问题（老师爱问）：

- “如果两个音同时弹，你怎么分？” → 承认这是复音问题，我们的 NNLS 是初步尝试，局限写在第 4 章。
- “你的系统函数 H(jω) 是什么？” → 指匹配滤波器那一支：H(ω)=S*(ω)e^{−jω(N−1)}。
- “零输入响应在你系统里体现在哪？” → 递推滤波器的初值为零，属于零状态响应；若讨论模板本身的谐波衰减，那是系统的自然响应（零输入）。

7.3 现场演示应急预案

故障	应急动作
麦克风没声音	立刻切“打开文件”，用预存的 `demo_phrase.wav`
环境太吵识别错	切到“低噪声模式”（提高门限）或直接演示预先录好的音频
程序崩溃	重启（提前练熟 15 秒内重启）；同时播放演示视频
电脑不兼容	提前把项目拷到 U 盘 + 云盘，用教室电脑跑 `run.bat`
老师要求现场弹琴	准备手机钢琴 App 作为音源，音量调大
投影颜色失真	界面配色提前在投影上试一次（深色背景 + 高对比大字）

附录 A 论文关键数据（复现时直接对照）

表 A-1 识别音域与基频（频率按 f = 440·2^((MIDI−69)/12) 计算，与论文表 1 一致至 0.01 Hz）

音符	频率/Hz	MIDI	音符	频率/Hz	MIDI	音符	频率/Hz	MIDI
B3	246.94	59	C4	261.63	60	C4#	277.18	61
D4	293.66	62	D4#	311.13	63	E4	329.63	64
F4	349.23	65	F4#	369.99	66	G4	392.00	67
G4#	415.30	68	A4	440.00	69	A4#	466.16	70
B4	493.88	71	C5	523.25	72	C5#	554.37	73
D5	587.33	74	D5#	622.25	75	E5	659.26	76
F5	698.46	77	F5#	739.99	78	G5	783.99	79
G5#	830.61	80	A5	880.00	81	A5#	932.33	82
B5	987.77	83	C6	1046.50	84	—	—	—

表 A-2 论文表 2 的相关矩阵（行 = 待测音符，列 = 标准音模板）

待测↓	C4	D4	E4	F4	G4	A4	B4
C4	**0.996**	0.014	0.007	0.026	0.107	0.014	0.021
D4	0.027	**0.999**	0.010	0.009	0.013	0.019	0.016
E4	0.013	0.014	**0.923**	0.080	0.022	0.014	0.024
F4	0.074	0.011	0.045	**0.932**	0.059	0.026	0.031
G4	0.140	0.029	0.031	0.039	**0.979**	0.051	0.023
A4	0.030	0.028	0.013	0.016	0.035	**0.909**	0.034

待测↓	C4	D4	E4	F4	G4	A4	B4
B4	0.049	0.016	0.019	0.012	0.012	0.023	**0.977**

表 A-3 论文表 3 的和弦分解结果

和弦组成	识别音名	与和弦的相关度
E4	E4	0.7062
C4	C4	0.5691
G4	G4	0.3763

表 A-4 论文表 4 的抗噪结果（复现目标）

SNR/dB	检测总次数	错检总次数	错检概率
20	200	0	0.0%
15	200	0	0.0%
10	200	0	0.0%
5	200	2	1.0%
0	200	9	4.5%

表 A-5 论文其余关键信息

项目	内容
论文题目	基于频域匹配滤波的音频信号分析与识别
作者	赵凌览、李晓峰（电子科技大学信息与通信工程学院）
期刊	实验科学与技术，2018，16(5)：34–38
DOI	10.3969/j.issn.1672-4550.2018.05.008
基金	国家自然科学基金（61671126）
测试乐句	《Mary Had a Little Lamb》前 7 音：E4 D4 C4 D4 E4 E4 E4（乐句共 20 音）
关键词	音频识别；端点检测；时频转换；匹配滤波

附录 B 报告提交前格式检查清单（逐条打勾）

☐ 纸张 A4、通栏排版

☐ 页边距：上 2.2 / 下 2.0 / 左 2.7 / 右 2.3 cm

☐ 页码居中，全文连续

☐ 行距：摘要与参考文献单倍，其余 1.2 倍

☐ 中文宋体、英文数字 Times New Roman（注意公式里的变量也要 TNR 斜体）

☐ 正文五号、摘要与文献小五号

☐ 一级标题小四粗体（段前 0.5 行），二级标题五号粗体

☐ 每段首行缩进 2 字符，段前段后无空行

☐ 图题在下、表题在上，编号连续，正文有“如图 3 所示”的交叉引用

☐ 所有图 300 dpi 以上，图内文字 ≥ 8 pt，中文不乱码

☐ 公式用公式编辑器，编号右对齐、连续

☐ 参考文献 ≥ 8 篇，按 GB/T 7714 格式，正文有引用标注

☐ 封面填全：题目、组号、任课教师、5 人学号姓名工作量比例签名、组长手机号

☐ 附录含核心代码与使用说明

☐ 查重率 < 20%

☐ 文件名与文件夹名符合“组号”要求

附录 C 每周周报模板（复制即用）

```
【第 X 周周报】组号：____    日期：____

一、本周完成（对照上周计划，逐项打勾）
  [完成]   成员 1：……
  [完成]   成员 2：……
  [未完成] 成员 3：……（原因：……）

二、本周验收标准达成情况
  P1 单音准确率：__%  （目标 ≥95%）→ 达成 / 未达成
  ……

三、下周计划（每人 ≤ 3 项，必须可验收）
  成员 1：①…… ②……
  ……

四、风险与需要支持
  R__：……  影响：……  需要谁：……

五、本周决策记录
  · 关于 ____ 的分歧，决定采用 ____，理由：____
```

最后一句给组长：你不需要是技术最强的人，你需要是那个**让约定清楚、让进度可见、让风险提前暴露**的人。第 1 周把 `types.py` 和信息表做出来，第 3 周把闭环跑通，第 6 周果断冻结——做到这三件事，这个组就不会差。